Problemas de DyV

Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027

Contenido

- Potencia
- Fibonacci
- Búsqueda binaria
- QuickSort
- MergeSort
- K-esimo

Potencia

Propiedades

$$a * a^{n-1}, \qquad a^n = \begin{cases} \left(a^{\frac{n}{2}}\right)^2, & n > 1, n\%2 == 0 \\ a * \left(a^{\frac{n}{2}}\right)^2, & n > 1, n\%2 == 1 \end{cases}$$

Algoritmos

$$a^n = \begin{cases} 1, & n = 0 \\ a, & n = 1 \\ a * a^{n-1}, & n > 1 \end{cases}$$

$$a^n = \begin{cases} 1, & n = 0 \\ a, & n = 1 \\ \left(a^{\frac{n}{2}}\right)^2, & n > 1, n\%2 = 0 \\ a * (a^{\frac{n}{2}})^2, & n > 1, n\%2 = 1 \end{cases}$$

Potencia

- No hay caso mejor y peor
- Primera opción → O(n)
- Segunda opción → O(log n)

Fibonacci

Asumiendo que la multiplicación y la suma son $\Theta(1)$

Propiedades:

$$f(n) = f(n-1) + f(n-2)$$

Algoritmos:

$$f(n) = \begin{cases} n, & 0 \le n \le 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

Complejidad sin memoria $\Theta\big(T(n)\big) > \Theta(\sqrt{2}^{n})$

Complejidad con memoria $\Theta\big(T(n)\big) = \Theta(n)$

Otra forma de hacerlo:

$$\begin{pmatrix} f_{n+1} \\ f_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} f_n \\ f_{n-1} \end{pmatrix}, \quad \begin{pmatrix} f_{n+1} \\ f_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$T(n) = T\left(\frac{n}{2}\right) + 1, \ \Theta\big(T(n)\big) = \Theta(\log n)$$

Aplicando el algoritmo de potencia visto y que multiplicar 2 matrices de 2x2 es $\Theta(1)$

Fibonacci

Asumiendo que la multiplicación y la suma son respectivamente $\Theta(d^2), \Theta(d)$. *Siendo d el número de dígitos del número y* $\Theta(Nd(f(n)) = \Theta(\log 2^n) = \Theta(n)$

$$f(n) = \begin{cases} n, & 0 \le n \le 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

El valor de un número *n* de Fibonnaci es de orden $\Theta(2^n)$ y el número de dígitos es de $\Theta(\log n)$, así, el número de dígitos de un número *n* de Fibonacci será de $\Theta(n)$

Complejidad sin memoria: $\Theta(T(n)) > \Theta(\sqrt{2}^n)$

Complejidad con memoria (sólo hace falta recordar los dos últimos elementos), si asumimos que la complejidad de la suma es $\Theta(d)$:

$$\sum_{i=0}^{n} i \cong_{\infty} \Theta(n^2)$$

Otra forma de hacerlo, si asumimos que la complejidad de la multiplicación es $\Theta(d^2)$

$$\begin{pmatrix} f_{n+1} \\ f_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} f_n \\ f_{n-1} \end{pmatrix}, \quad \begin{pmatrix} f_{n+1} \\ f_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Usando el algoritmo de la potencia entera para la potencia de la matriz:

$$T(n) = T\left(\frac{n}{2}\right) + n^2, \ \Theta(T(n)) = \Theta(n^2)$$

La multiplicación puede ser de orden $\Theta(d^{1.58})$ *(Karatsuba) o* $\Theta(d^{1.46})$ *(Toom-Cook3) =>* $\Theta(T(n)) \cong \Theta(n^{1.5})$

Búsqueda binaria

La búsqueda binaria es un **algoritmo eficiente para encontrar un elemento en una lista ordenada de elementos**

Búsqueda binaria

```
private static <E> int bSearch(List<E> lista, int i, int j,
        E key, Comparator<? super E> ord){
    int r;
    int k;
    if(j-i == 0){
        r = -1;
    }else{
        k = (i+j)/2;
        int r1 = ord.compare(key,lista.get(k));
        if(r1==0){
            r = k;
        }else if(r1<0){
            r = bSearch(lista,i,k,key,ord);
        }else{
            r = bSearch(lista,k+1,j,key,ord);
        }
    }
    return r;
}
```

¿tamaño?

¿caso mejor y peor?

¿T(n)?

Búsqueda binaria

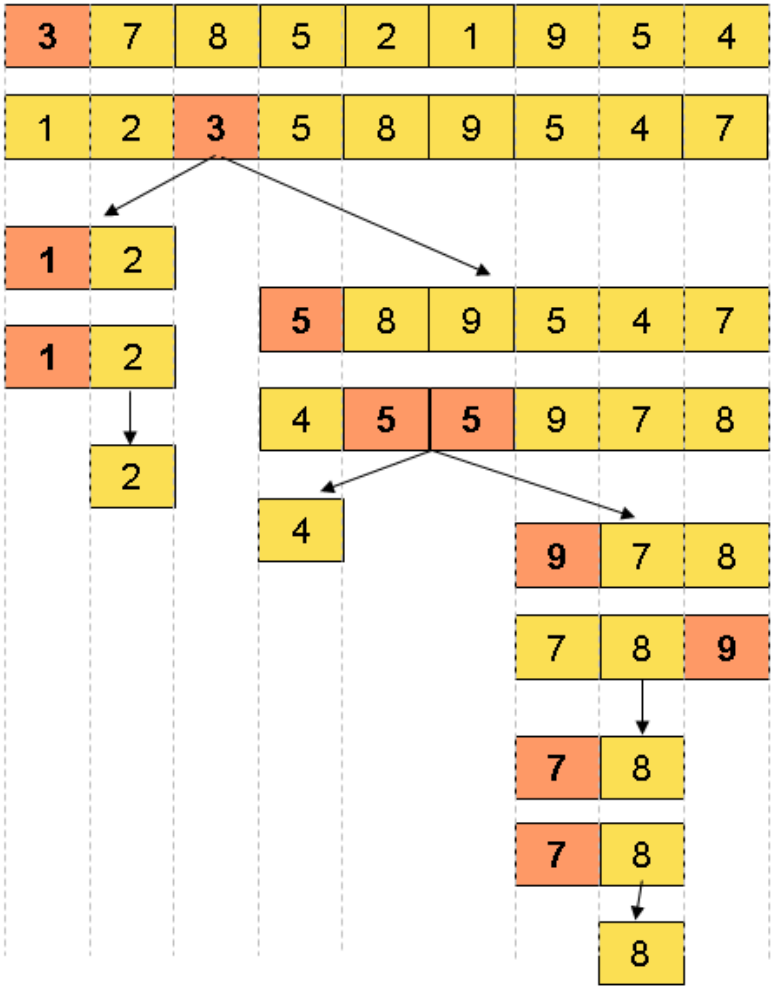
- Tamaño → n = j - i
- Caso mejor: divido y encuentro el elemento que busco → O(1)
- Caso peor: divido de forma recursiva hasta quedarme con un solo elemento o no lo encuentro → O(log n)

QuickSort

Algoritmo de ordenación

```
private static <E> void quickSort(List<E> ls, Integer i,
      Integer j, Comparator<? super E> ord){
   Preconditions.checkArgument(j>=i);
   if(j-i <= THRESHOLD ){
      sortBaseCase(ls, i, j, ord);
   }else{
      E pivot = choosePivot(ls, i, j);
      IntPair p = dutchFlag(ls, pivot, i, j, ord);
      quickSort(ls,i,p.first(),ord);
      quickSort(ls,p.second(),j,ord);
   }
}
```

QuickSort



QuickSort

Algoritmo de ordenación

```
private static <E> void quickSort(List<E> ls, Integer i,
      Integer j, Comparator<? super E> ord){
   Preconditions.checkArgument(j>=i);
   if(j-i <= THRESHOLD ){
      sortBaseCase(ls, i, j, ord);
   }else{
      E pivot = choosePivot(ls, i, j);
      IntPair p = dutchFlag(ls, pivot, i, j, ord);
      quickSort(ls,i,p.first(),ord);
      quickSort(ls,p.second(),j,ord);
   }
}
```

¿caso mejor y peor?

¿T(n)?

QuickSort

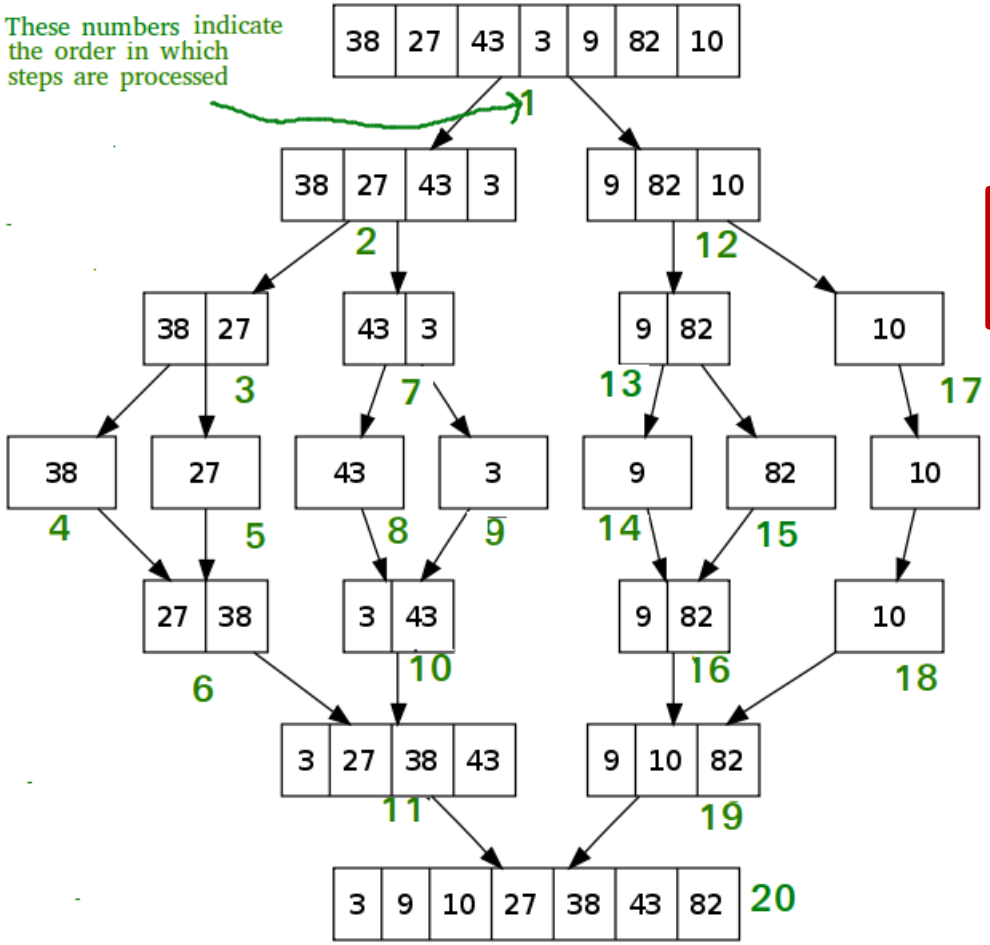
- Tamaño = n = j - i
- La elección del pivote es crítica:
  - Escogiendo el pivote más o menos centrado → $\Theta(n \log n)$ en el caso medio.
  - Mala elección del pivote → $\Theta(n^2)$ (por ejemplo, escoger el primer o el último elemento y que la secuencia estuviera inicialmente ordenada).

MergeSort

Algoritmo de ordenación

```
private static <E> void mgSort(List<E> ls1, Integer i, Integer j,
      Comparator<? super E> ord, List<E> ls2, Integer threshold){
   if(j-i <= threshold){
      sortBaseCase(ls1, i, j, ord);
   }
   else {
      int k = (j+i)/2;
      mgSort(ls1,i,k,ord,ls2,threshold);
      mgSort(ls1,k,j,ord,ls2,threshold);
      merge(ls1,i,k,ls1,k,j,ls2,i,j,ord);
      copy(ls1,i,j,ls2);
   }
}
```

Merge Sort



¿tamaño?

¿caso mejor y peor?

¿T(n)?

Merge Sort

- Tamaño → n = j - i
- De O(n log n) para el caso mejor, peor y medio.

K-esimo

Devuelve el **k-ésimo menor** valor de un conjunto de datos

- Sólo nos interesa ese valor
- ¿Cómo hacerlo eficientemente?

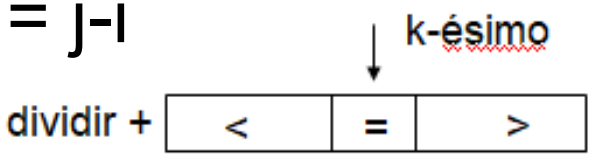
K-esimo

```
private static <E> E chooseKth(List<E> ls, Integer i,
      Integer j, Integer k, Comparator<? super E> ord){
   Preconditions.checkArgument(j>=i);
   E r;
   if(j-i == 0){
      r = null;
   } else if(j-i == 1){
      r = ls.get(i);
   }else{
      E pivot = choosePivot(ls, i, j);
      IntPair p = dutchFlag(ls, pivot, i, j, ord);
      if(k < p.first()){
         r = chooseKth(ls,i,p.first(),k,ord);
      }else if(k >= p.second()){
         r = chooseKth(ls,p.second(),j,k,ord);
      }else{
         r = ls.get(k);
      }
   }
   return r;
}
```

¿caso mejor y peor?

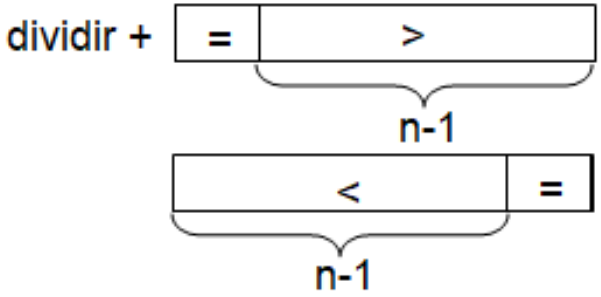
K-esimo

- Tamaño → n = j-i
- Caso mejor:



$T(n) \in O(n)$

- Caso peor:



Nota: El k-esimo con pseudomediana sale O(n) en el caso peor.

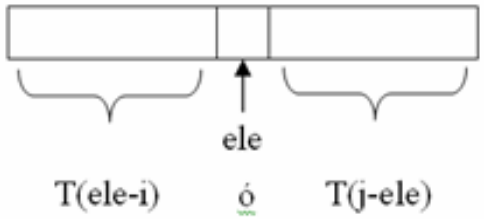
$$T(n) = dn + T(n-1) \in O(n^2)$$

- Caso medio:

donde l=u=v, puesto que los datos son distintos.



$$T(n) = dn + \frac{1}{n}\sum_{l=i}^{j} \max(T(l-i), T(j-l)) \in O(n)$$



Problemas de DyV

Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027